



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Desarrollo de un Agente Conversacional basado en IA para la Gestión Burocrática en la Universidad de Sevilla

Realizado por

Kevin Amador Calzadilla

Para la obtención del título de

Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por

José Enrique Sánchez López y Pablo Reina Jiménez

En el departamento de

Lenguajes y Sistemas Informáticos

15 de abril de 2026

Índice general

I	Introducción y Conceptos	1
1.	Introducción	2
1.1.	Marco conceptual	2
1.2.	Motivación	2
1.3.	Objetivos	3
1.3.1.	Objetivo general	3
1.3.2.	Objetivos técnicos	3
1.4.	Alcance del proyecto	4
1.5.	Estructura del proyecto	5
2.	Estado del Arte	6
2.1.	Modelos de Lenguaje Grande (LLMs)	6
2.1.1.	Definición y evolución	6
2.2.	Generación Aumentada por Recuperación (RAG)	7
2.2.1.	Concepto y arquitectura general	7
2.2.2.	Embeddings y bases de datos vectoriales	8
2.2.3.	Estrategias de fragmentación (chunking)	8
2.3.	Agentes de Inteligencia Artificial	9
2.3.1.	De LLMs a sistemas agénticos	9
2.3.2.	Grafos de estado: LangGraph	9
2.4.	Chatbots institucionales: antecedentes	10
2.5.	Tecnologías clave del ecosistema	10
2.5.1.	Lenguaje y Frameworks base	10
2.5.2.	Orquestación y Lógica Agéntica	11
2.5.3.	Pinecone como base de datos vectorial	11
2.5.4.	Cohere: embeddings y reranking	11
2.5.5.	Groq: inferencia de baja latencia	11
2.5.6.	LlamaParse: procesamiento de documentos	11
2.5.7.	Encriptado de contraseñas	12
2.5.8.	Gestión de la base de datos relacional	12

II	Metodología y Planificación	13
3.	Metodología de desarrollo	14
3.1.	Ciclo de vida del proyecto	14
3.2.	Metodología Scrum	14
3.2.1.	Roles	14
3.2.2.	Artefactos	15
3.2.3.	Ceremonias	15
3.3.	Herramientas de gestión	15
4.	Planificación	16
4.1.	Definición de sprints	16
4.2.	Diagrama de Gantt	17
4.3.	Estimación de esfuerzo	17
4.4.	Análisis de riesgos	17
4.5.	Estimación de costes	18
4.5.1.	Costes de personal	18
4.5.2.	Costes de hardware y software	18
4.5.3.	Costes indirectos	19
4.5.4.	Coste total del proyecto	19
III	Análisis y Diseño	20
5.	Análisis de requisitos	21
5.1.	Requisitos funcionales	21
5.2.	Requisitos no funcionales	22
5.3.	Casos de uso	22
6.	Diseño de la arquitectura	24
6.1.	Arquitectura general del sistema	24
6.2.	Arquitectura del backend (FastAPI)	24
6.2.1.	Modelo de datos	24
6.2.2.	Endpoints de la API REST	25
6.2.3.	Autenticación y autorización (JWT)	25
6.3.	Diseño del agente conversacional	26
6.3.1.	Grafo de estados (LangGraph)	26
6.3.2.	Nodo Estado Inicial	26
6.3.3.	Nodo Router: clasificación de intenciones	26
6.3.4.	Nodo Recuperador: pipeline RAG	26
6.3.5.	Nodo Entrevistador: gestión de ambigüedades	27

6.3.6. Nodos Resolutores: respuestas especializadas	27
6.3.7. Nodo Rechazo Amable	27
6.4. Diseño del sistema Dual RAG	27
6.5. Diseño del frontend (Streamlit)	28

IV Implementación 29

7. Infraestructura y despliegue 30

7.1. Contenedorización con Docker	30
7.1.1. Docker Compose: orquestación de servicios	30
7.1.2. Configuración de red y variables de entorno	30
7.2. Base de datos PostgreSQL	30

8. Pipeline de ingesta de documentos 32

8.1. Procesamiento con LlamaParse	32
8.2. Fragmentación semántica (chunking)	32
8.2.1. Evolución de la estrategia de fragmentación	32
8.2.2. El problema del doble splitter	33
8.2.3. Configuración final: MarkdownTextSplitter	33
8.3. Generación de embeddings con Cohere	33
8.4. Almacenamiento en Pinecone	34

9. Implementación del agente 35

9.1. Esquema de estado (StateSchema)	35
9.2. Sistema de prompts	35
9.2.1. Prompt de detección de intención	35
9.2.2. Prompt de reformulación	36
9.2.3. Prompt de suficiencia de información	36
9.2.4. Prompt clasificador de categorías	36
9.2.5. Prompts de los nodos resolutores	36
9.3. Pipeline de recuperación	36
9.3.1. Búsqueda semántica en Pinecone	36
9.3.2. Reranking con Cohere	37
9.4. Gestión de memoria conversacional	37
9.4.1. Ventana deslizante de mensajes	37
9.4.2. Compresión de memoria con resumen	37
9.5. Generación automática de títulos	37

10.Implementación del frontend	38
10.1. Interfaz de autenticación	38
10.2. Panel de conversaciones	38
10.3. Interfaz de chat	38
10.4. Panel de administración	39
11.Selección y configuración de modelos	40
11.1. Evolución de la infraestructura de inferencia	40
11.1.1. Fase 1: Modelos locales (Llama 3.1 8B)	40
11.1.2. Fase 2: API de HuggingFace	40
11.1.3. Fase 3: API de Groq	40
11.2. Estrategia multi-modelo	41
11.2.1. LLM clasificador (Llama 3.1 8B Instant)	41
11.2.2. LLM ligero (Llama 3.1 8B Instant)	41
11.2.3. LLM principal (Llama 3.3 70B Versatile)	41
11.3. Ajuste de hiperparámetros	41
V Evaluación y Validación	42
12.Estrategia de evaluación	43
12.1. Dataset de evaluación	43
12.2. Métricas empleadas	43
12.2.1. Routing Accuracy	43
12.2.2. Faithfulness	44
12.2.3. Answer Relevancy	44
12.2.4. Correctness	44
12.3. Evaluación con LLM como juez	44
13.Resultados	45
13.1. Precisión del enrutamiento	45
13.2. Análisis de fidelidad (Faithfulness)	45
13.3. Análisis de relevancia	46
13.4. Análisis de corrección	46
13.5. Comparativa: entorno local vs Groq	46
13.6. Evaluación del retriever (Hit Rate)	46
VI Conclusiones y Trabajo Futuro	48
14.Conclusiones	49

14.1. Cumplimiento de objetivos	49
14.2. Lecciones aprendidas	49
14.3. Dificultades encontradas	50
15.Trabajo futuro	51
15.1. Mejoras en el pipeline RAG	51
15.2. Ampliación de la base de conocimiento	51
15.3. Integración con canales de mensajería	51
15.4. Despliegue en producción	51
A. Batería de validación completa	55
B. Prompts del sistema	56
C. Uso de la Inteligencia Artificial	57

Índice de figuras

2.1. Estructura típica de un sistema RAG	8
4.1. Diagrama de Gantt de la planificación del TFG	17

Índice de tablas

4.1. Planificación de sprints del proyecto	16
4.2. Matriz de análisis de riesgos del proyecto	18
4.3. Presupuesto total estimado	19
5.1. Requisitos funcionales del sistema	21
5.2. Requisitos no funcionales del sistema	22
6.1. Endpoints principales de la API	25
12.1. Distribución del dataset de evaluación por categoría	43
13.1. Resultados de la precisión de enrutamiento	45
13.2. Comparativa entre entorno local y Groq	46

Parte I

Introducción y Conceptos

Capítulo 1

Introducción

1.1. Marco conceptual

En los últimos años, los Modelos de Lenguaje Grandes (LLMs) han revolucionado la forma en que las máquinas entienden y generan texto. Sin embargo, cualquiera que haya trabajado con ellos sabe que tienen un grave problema, las llamadas “alucinaciones”. Básicamente, el modelo puede generar respuestas que suenan perfectamente razonables pero que, en realidad, se las ha inventado por completo. Esto los convierte en herramientas poco fiables cuando necesitamos precisión, algo imprescindible si hablamos de la gestión de documentación universitaria.

Para lidiar con este problema, en este trabajo se ha optado por implementar un sistema RAG (*Retrieval-Augmented Generation*). La idea es sencilla pero eficaz: en lugar de pedirle al modelo que “recuerde” la información, le damos acceso directo a la documentación oficial, previamente procesada y convertida en vectores. De esta manera, cada respuesta que genera combina la fluidez conversacional propia de un LLM con datos reales extraídos de fuentes institucionales verificables.

Nota: Estos conceptos y tecnologías base se definirán en profundidad en el Capítulo 2.

1.2. Motivación

Creo que cualquier persona que haya pasado por una universidad grande sabe lo desesperante que puede llegar a ser encontrar un dato concreto entre la maraña de documentación institucional. En mi caso, como estudiante de la Universidad de Sevilla, he vivido en primera persona esa frustración más veces de las que me gustaría admitir: buscar un plazo de matrícula que aparece en un PDF de 80 páginas, intentar descifrar los criterios de un baremo de contratación o simplemente averiguar a qué ventanilla acudir para resolver un trámite. Y no soy el único: compañeros, profesores e incluso el propio personal administrativo se enfrentan a diario al mismo problema.

Lo habitual cuando surge una duda administrativa es descargar varios PDFs extensos y poco manejables, preguntar en la secretaría de la facultad —si es que coincide con el horario de atención— o enviar un correo electrónico sabiendo que la respuesta puede tardar días. Este desgaste no lo sufre solo el usuario, que puede perder horas buscando algo que debería encontrar en minutos; también lo padece el Personal de Administración y Servicios (PAS), cuya jornada se consume en buena parte respondiendo las mismas dudas una y otra vez, dudas que teóricamente ya están resueltas en los documentos publicados.

Ahí es precisamente donde nace la motivación de este proyecto: contar con un asistente virtual disponible las 24 horas, capaz de resolver consultas de forma inmediata, precisa y fundamentada en la documentación real de la universidad. A través de una interfaz sencilla, el sistema interpreta lo que el usuario necesita en lenguaje natural y le devuelve una respuesta clara, sin obligarle a bucear entre decenas de documentos.

1.3. Objetivos

El propósito de este proyecto es, en esencia, cambiar por completo la forma en que los usuarios interactúan con la densa normativa de la Universidad de Sevilla, apoyándonos para ello en tecnologías punteras del campo de la Inteligencia Artificial.

1.3.1. Objetivo general

Lo que se busca con este trabajo es diseñar, desarrollar y poner a prueba un asistente conversacional inteligente basado en arquitecturas RAG (*Retrieval-Augmented Generation*). Este asistente debe ser capaz de ayudar a candidatos, estudiantes y docentes apoyándose únicamente en los documentos que los administradores le proporcionan. La clave está en que pueda interpretar consultas complejas sobre trámites burocráticos y normativos, y devolver respuestas lo suficientemente precisas como para que el usuario no tenga que recurrir a otras fuentes.

1.3.2. Objetivos técnicos

Para poder alcanzar ese objetivo general, se han marcado una serie de hitos técnicos concretos:

- **Optimización de la ingesta de documentos:** Desarrollar un flujo de procesamiento mediante el que es posible interpretar documentos con estructuras complejas (tablas, anexos, notas al pie) mediante LlamaParse, garantizando que la información no pierda sentido al ser vectorizada.
- **Estructura agéntica:** Implementar un sistema basado en LangGraph con nodos especializados para la resolución de consultas:

- Nodos resultadores expertos para respuestas directas.
 - Nodo entrevistador para resolución de ambigüedades.
 - Nodo de rechazo para consultas fuera de dominio.
- **Diseño de un sistema dual RAG:** Construir una arquitectura capaz de separar el conocimiento normativo de las directrices de estilo y buenas prácticas cuando se atiende a un usuario
 - **Minimizar el tiempo de respuesta:** Reducir los tiempos de respuesta a menos de veinte segundos combinando el uso de modelos pesados para generar la respuesta final con modelos livianos para las decisiones intermedias.
 - **Validación del sistema:** Evaluar el sistema con varios casos de uso reales que simulen conversaciones con estudiantes, docentes y personal administrativo. Los criterios de evaluación se basarán en métricas de *Faithfulness*, *Relevancy* y *Correctness*.

1.4. Alcance del proyecto

Conviene dejar claro desde el principio qué abarca este proyecto y qué queda fuera. En concreto, el trabajo incluye:

- El desarrollo de un **backend API REST** capaz de gestionar las peticiones de usuarios, el historial de conversaciones y toda la lógica del agente.
- La implementación de un **agente conversacional** basado en LangGraph con capacidad de enrutamiento inteligente y recuperación semántica de documentos.
- Un **pipeline de ingesta** que procesa documentos PDF oficiales de la US y los almacena como vectores en Pinecone.
- Una **interfaz web** desarrollada con Streamlit que permite la interacción con el agente en tiempo real.
- La **contenedorización** completa del sistema mediante Docker Compose.
- Una **evaluación cuantitativa** del sistema.

Queda fuera del alcance de este TFG el despliegue en un entorno de producción con alta disponibilidad, la integración con sistemas internos de la US (como SEVIUS o la Secretaría Virtual) y la creación de un sistema de retroalimentación de usuarios. Estas líneas, no obstante, se retoman como trabajo futuro en el capítulo final.

1.5. Estructura del proyecto

Para que la lectura de este documento resulte lo más fluida posible, se ha organizado en seis bloques temáticos bien diferenciados:

- **Parte I: Introducción y Conceptos.** Se expone la motivación del trabajo, los objetivos perseguidos y se realiza un repaso por el estado del arte de las tecnologías implicadas, incluyendo los LLMs, sistemas RAG y arquitecturas agénticas.
- **Parte II: Metodología y Planificación.** Describe el ciclo de vida elegido para el proyecto, detallando la adaptación del marco de trabajo Scrum y proporcionando el desglose temporal de las tareas.
- **Parte III: Análisis y Diseño.** Desglosa los requisitos funcionales y no funcionales, e introduce la arquitectura del sistema tanto a nivel conceptual (LangGraph) como de servicios (FastAPI, Streamlit, bases de datos).
- **Parte IV: Implementación.** Profundiza en los detalles técnicos del desarrollo: cómo se resolvieron los problemas de ingesta de documentos, la orquestación del agente y los desafíos surgidos con los LLM y su latencia.
- **Parte V: Evaluación y Validación.** Muestra los resultados obtenidos tras someter al agente a una exhaustiva batería de pruebas, utilizando un LLM como juez para evaluar su rigor y fiabilidad.
- **Parte VI: Conclusiones y Trabajo Futuro.** Recopila las reflexiones finales, valorando si se cumplieron los objetivos iniciales, y plantea posibles vías para continuar mejorando el sistema.

Capítulo 2

Estado del Arte

2.1. Modelos de Lenguaje Grande (LLMs)

2.1.1. Definición y evolución

Un Modelo de Lenguaje de Gran Escala (LLM, por sus siglas en inglés) es un sistema de *Deep Learning* fundamentado en redes neuronales con miles de millones de parámetros. Su entrenamiento principal consiste en asimilar cantidades masivas de texto sin etiquetar para aprender a predecir, estadísticamente, cuál es la siguiente palabra en una secuencia. El verdadero salto cualitativo de estos sistemas llegó con la introducción del mecanismo de *atención*, presentado en el influyente artículo 'Attention is all you need' [12]. Esta arquitectura, conocida como *Transformer*, permite al modelo sopesar qué partes del texto de entrada son más relevantes, procesando la información en paralelo y capturando el contexto a largo plazo de forma mucho más eficaz que sus predecesores.

Si echamos la vista atrás, desde los primeros modelos como GPT-2 en 2019 hasta las iteraciones actuales como Llama 3 [14], GPT-4 o Claude, el tamaño de estas redes ha crecido de forma desmesurada. Curiosamente, este aumento exponencial de parámetros ha revelado un fenómeno fascinante: al alcanzar cierta escala, los modelos desarrollan habilidades emergentes que nadie programó explícitamente durante su entrenamiento, tales como el razonamiento deductivo, la escritura de código o la capacidad de seguir instrucciones complejas.

Dicho todo esto, y a pesar de lo impresionantes que resultan, los LLMs arrastran limitaciones que conviene tener muy presentes:

- **Alucinaciones:** Quizá el problema más conocido. El modelo puede generar información que suena perfectamente creíble pero que es sencillamente falsa. Al fin y al cabo, lo que hace es predecir el siguiente token en función de patrones estadísticos; no tiene forma de “saber” si lo que dice es cierto o no.
- **Conocimiento congelado:** Todo lo que un LLM sabe quedó fijado en el momento

de su entrenamiento. No puede acceder por su cuenta a información nueva ni consultar documentos específicos de un ámbito concreto, a menos que alguien se los proporcione explícitamente.

- **Ventana de contexto limitada:** Aunque las ventanas de contexto han ido creciendo año tras año, el espacio del que dispone el modelo para “leer” información dentro de un prompt sigue siendo finito. Esto obliga a ser selectivos con qué le pasamos y cómo.

Estas carencias son las que han empujado a la comunidad investigadora a buscar técnicas complementarias que las mitiguen. En este trabajo nos centraremos en dos de ellas: los sistemas de Generación Aumentada por Recuperación (RAG) y las arquitecturas de agentes conversacionales.

2.2. Generación Aumentada por Recuperación (RAG)

2.2.1. Concepto y arquitectura general

La Generación Aumentada por Recuperación (*Retrieval-Augmented Generation*, RAG) es un paradigma propuesto por Lewis et al. [13] que combina la capacidad generativa de los LLMs con un sistema de recuperación de información. En lugar de confiar exclusivamente en el conocimiento paramétrico del modelo, RAG inyecta fragmentos de documentos relevantes en el prompt antes de la generación, permitiendo que la respuesta se fundamente en fuentes verificables.

El flujo típico de un sistema RAG consta de tres fases:

1. **Ingesta:** Los documentos se procesan, fragmentan y transforman en representaciones vectoriales (embeddings) que se almacenan en una base de datos vectorial.
2. **Recuperación:** Ante una consulta del usuario, se genera el embedding de la pregunta y se buscan los fragmentos más similares semánticamente en la base de datos.
3. **Generación:** Los fragmentos recuperados se inyectan como contexto en el prompt del LLM, que genera una respuesta basada en dicha información.

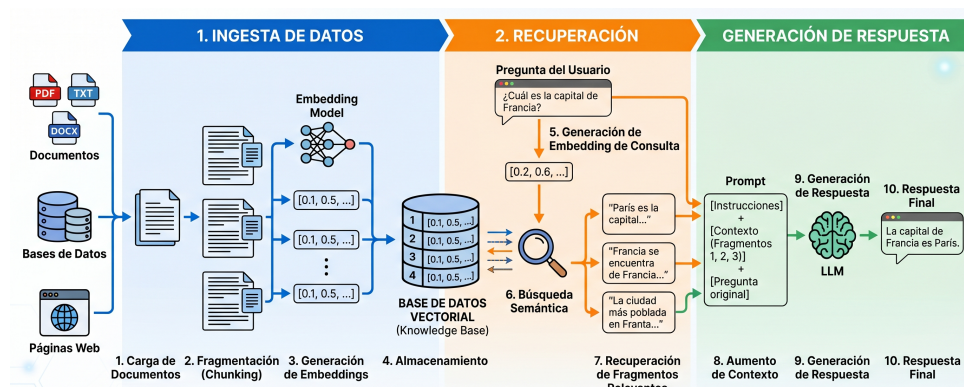


Figura 2.1: Estructura típica de un sistema RAG

2.2.2. Embeddings y bases de datos vectoriales

Un *embedding* es una representación numérica densa de un fragmento de texto en un espacio vectorial de alta dimensionalidad. Dos textos semánticamente similares producirán vectores cercanos en este espacio, lo que permite realizar búsquedas por similitud semántica en lugar de por coincidencia exacta de palabras.

Los modelos de embeddings multilingües son capaces de generar representaciones que capturan el significado del texto independientemente del idioma, lo que resulta esencial para un sistema que opera sobre documentación en español.

Las bases de datos vectoriales, como Pinecone, están optimizadas para almacenar y buscar eficientemente entre millones de vectores. Estas utilizan algoritmos de búsqueda aproximada de vecinos más cercanos (ANN, *Approximate Nearest Neighbors*) que permiten recuperar los documentos más relevantes en milisegundos.

2.2.3. Estrategias de fragmentación (chunking)

La fragmentación o *chunking* es el proceso de dividir documentos extensos en segmentos manejables para su vectorización. La calidad de esta fragmentación impacta directamente en la precisión de la recuperación, y existen diversos tipos como son los siguientes:

- **Fragmentación por tamaño fijo:** Divide el texto en bloques de n caracteres con un solapamiento (*overlap*) configurable. Es simple pero puede romper el contexto de tablas o párrafos.
- **Fragmentación semántica:** Respeta los límites naturales del documento (encabezados, párrafos, tablas) para preservar la coherencia. Herramientas como `MarkdownTextSplitter` de LangChain utilizan la estructura Markdown como guía para los cortes.
- **Fragmentación asistida por IA:** Herramientas como LlamaParse utilizan modelos de visión para interpretar la estructura del documento, preservando tablas, notas al pie y jerarquías de encabezados.

2.3. Agentes de Inteligencia Artificial

2.3.1. De LLMs a sistemas agénticos

Si nos paramos a pensarlo, un LLM por sí solo funciona como una caja que recibe texto y devuelve texto. Para lo que buscamos en este proyecto —un asistente que sea capaz de tomar decisiones, consultar fuentes y mantener el hilo de una conversación— eso se queda corto. Necesitamos un paso más: convertir ese modelo en el “cerebro” de un agente de IA que, además de generar lenguaje, pueda hacer lo siguiente:

- **Razonar:** Analizar la consulta del usuario y decidir las acciones que debe tomar.
- **Planificar:** Descomponer tareas complejas en subtareas para poder resolverlas de manera más eficiente.
- **Actuar:** Invocar herramientas externas, mediante las que enriquecer la calidad de la información a la que el modelo puede acceder (APIs, bases de datos, buscadores, etc).
- **Recordar:** Mantener un estado conversacional a lo largo de múltiples interacciones con el usuario.

2.3.2. Grafos de estado: LangGraph

Una vez decidido que íbamos a trabajar con agentes, hacía falta una herramienta que nos permitiera orquestar todo esto desde Python. Después de evaluar varias alternativas, se optó por LangGraph, una extensión de LangChain que modela el flujo de un agente como un grafo dirigido de estados. Cada operación se representa como un nodo del grafo, y las transiciones condicionales entre operaciones se definen como aristas. Este planteamiento ofrece varias ventajas claras frente al esquema clásico de entrada/salida:

- **Flujos condicionales:** En función de la entrada del usuario, el agente podrá tomar caminos diferentes teniendo a su disposición diversas herramientas.
- **Ciclos controlados:** Permite implementar bucles controlados de interacción con el usuario en los cuales el agente pueda solicitar información adicional en caso de ser algo ambigua la consulta.
- **Estado compartido:** Todos los nodos acceden y modifican a un estado global tipado (`StateSchema`), garantizando la coherencia.
- **Depuración:** Debido a la estructura de grafo facilita la trazabilidad de las decisiones del agente, facilitando de esta manera la identificación de errores.

2.4. Chatbots institucionales: antecedentes

La idea de poner un chatbot en una universidad no es nada nueva. Desde hace años, muchas instituciones han desplegado asistentes virtuales apoyándose en motores de Reconocimiento del Lenguaje Natural (NLU) y clasificación de intenciones, con plataformas conocidas como DialogFlow o Rasa. Estas herramientas funcionan razonablemente bien cuando el flujo de diálogo está bien acotado, pero se quedan cortas en cuanto el volumen de información crece. El motivo es que se basan en árboles de decisión y reglas predefinidas, lo que obliga a un mantenimiento constante: cada vez que surge una nueva casuística hay que añadir frases de entrenamiento y mapear la interacción manualmente.

Lo que diferencia a este trabajo de esos enfoques clásicos es el salto de un modelo basado en intenciones a uno generativo y dinámico. Combinando la flexibilidad de los LLMs con el conocimiento institucional actualizado mediante RAG y una estructura agéntica con distintos nodos especializados, el sistema es capaz de localizar la respuesta en la documentación oficial y presentarla de forma clara, sin necesidad de haber previsto cada posible pregunta de antemano.

2.5. Tecnologías clave del ecosistema

2.5.1. Lenguaje y Frameworks base

Como lenguaje principal se ha utilizado Python. La elección, siendo sincero, fue bastante natural: a día de hoy es prácticamente el estándar de facto en todo lo que rodea a la Inteligencia Artificial. Su ecosistema de bibliotecas es enorme y ofrece compatibilidad directa con las herramientas de orquestación que veremos a continuación (LangChain, LangGraph), además de con prácticamente cualquier librería de procesamiento de datos que se necesite.

Sobre esta base, el sistema se ha estructurado utilizando dos *frameworks* principales, ambos orientados en cada una de las capas de nuestra aplicación (frontend/backend):

- **FastAPI:** para la implementación de la lógica del servidor *backend* y la exposición de servicios mediante una API REST. Esta tecnología ha sido seleccionada puesto que destaca por su alto rendimiento y su soporte nativo ante operaciones asíncronas, característica crítica para gestionar de manera eficiente los tiempos de espera derivados de las llamadas a los modelos y a la base de datos vectorial, evitando así en todo momento el bloqueo del hilo principal de ejecución.
- **Streamlit:** permite una construcción ágil de aplicaciones web interactivas orientadas a datos directamente en Python. Se ha tomado esta opción debido a que

incorpora componentes nativos diseñados específicamente para interfaces conversacionales. Facilitando enormemente la implementación del historial del chat, la gestión del estado de la sesión en el cliente y la renderización de las respuestas del agente de forma progresiva (*streaming*)

2.5.2. Orquestación y Lógica Agéntica

LangChain [2] es un framework de código abierto que proporciona abstracciones para conectar LLMs con fuentes de datos externas, cadenas de procesamiento y herramientas. LangGraph [3] extiende este framework con la capacidad de definir grafos de estado, permitiendo flujos de control complejos con nodos y aristas condicionales.

2.5.3. Pinecone como base de datos vectorial

Pinecone [5] es un servicio gestionado de base de datos vectorial que ofrece almacenamiento, indexación y búsqueda de vectores de alta dimensionalidad con baja latencia. Su modelo serverless elimina la necesidad de gestionar infraestructura, lo que simplifica el despliegue.

2.5.4. Cohere: embeddings y reranking

Cohere [6] proporciona dos servicios fundamentales para este proyecto: un modelo de embeddings multilingüe (`embed-multilingual-v3.0`) para la vectorización de documentos en español, y un modelo de reranking (`rerank-v4.0-pro`) que reordena los resultados de la búsqueda vectorial según su relevancia contextual con la consulta.

2.5.5. Groq: inferencia de baja latencia

Groq [7] es una plataforma de inferencia que utiliza hardware especializado (LPUs, *Language Processing Units*) diseñado específicamente para la ejecución de modelos de lenguaje. Frente a las GPUs tradicionales, los LPUs ofrecen latencias de inferencia significativamente menores, lo que resulta crítico para un asistente conversacional en tiempo real.

2.5.6. LlamaParse: procesamiento de documentos

LlamaParse [8] es un servicio de procesamiento de documentos que utiliza modelos de visión por computador para extraer contenido de archivos PDF preservando su estructura original. A diferencia de extractores de texto convencionales, LlamaParse es capaz de interpretar tablas complejas, notas al pie y jerarquías de encabezados, convirtiendo el contenido a formato Markdown estructurado.

2.5.7. Encriptado de contraseñas

Para poder garantizar la seguridad y privacidad de los usuarios, se ha implementado un flujo de *hashing* para las contraseñas, utilizando el algoritmo bcrypt. Se ha elegido principalmente porque, además de ofrecer un cifrado unidireccional (irrecuperable), incorpora de forma nativa un sistema de *salting*. Este consiste en añadir datos aleatorios a la contraseña antes de cifrarla, lo que permite ajustar el 'coste' computacional del algoritmo. Esto hace que el sistema sea resistente a ataques de fuerza bruta, garantizando que, incluso ante una filtración de la base de datos, los atacantes no puedan obtener las contraseñas reales.

2.5.8. Gestión de la base de datos relacional

Para la persistencia de los datos, se plantearon diversas opciones, tales como MySQL, SQLServer u Oracle. Para el objetivo de este proyecto, se ha considerado que la opción más conveniente es PostgreSQL, puesto que no solo ofrece una buena solución, sino que además ya se tenía algo de experiencia previa trabajando con la misma. Se trata de un sistema de gestión de base de datos relacional que se caracteriza por:

- **Integridad y fiabilidad de datos:** garantiza el cumplimiento estricto de las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), lo que le permite ser muy tolerante a fallos, garantizando así transacciones muy seguras.
- **Código abierto y gratuito:** a diferencia de las bases de datos que requieren de una licencia comercial para poder hacer uso de todas sus funciones, esta tecnología al ser *open source* no tiene ningún tipo de coste, convirtiéndola en una alternativa rentable frente a opciones privativas.
- **Rendimiento y escalabilidad:** está diseñado para manejar grandes volúmenes de datos y cargas de trabajo pesadas, lo que le permite ser altamente escalable y eficiente en consultas complejas.

Para poder interaccionar con la base de datos desde el backend, aunque se valoró el uso de *psycopg2*, la cual ofrece una gran velocidad a la hora de resolver tareas sencillas, se requería de un ORM (Object-Relational Mapping) más robusto. Finalmente, se optó por *SQLAlchemy* dado que ofrece tanto el propio ORM como un *SQL Expression Language*, facilitando enormemente la realización de consultas. Además, destaca por su facilidad para gestionar conexiones, migraciones y proteger el sistema frente a vulnerabilidades como la inyección SQL.

Parte II

Metodología y Planificación

Capítulo 3

Metodología de desarrollo

3.1. Ciclo de vida del proyecto

Desde el primer momento tuve claro que un enfoque en cascada no iba a funcionar para este proyecto. Al tratarse de un sistema donde las decisiones sobre modelos, estrategias de fragmentación o hiperparámetros dependen en gran medida de resultados empíricos —es decir, de probar y ver qué sale—, necesitaba un ciclo de vida que me permitiera equivocarme pronto y corregir rápido. Por eso se optó por un enfoque iterativo e incremental, en el que cada iteración produce un incremento funcional del sistema.

En la práctica, cada iteración ha seguido el ciclo: análisis → diseño → implementación → pruebas → retroalimentación. Esto me permitió ir refinando componentes críticos como el pipeline de ingesta o la configuración de los modelos de lenguaje a medida que aprendía de los errores.

3.2. Metodología Scrum

Como marco de trabajo ágil se adoptó Scrum, aunque evidentemente hubo que adaptarlo a la realidad de un TFG individual. No tiene mucho sentido hacer una *daily* contigo mismo, pero sí que resultó muy útil mantener la filosofía de sprints cortos, backlog priorizado y revisiones periódicas.

3.2.1. Roles

Al ser un proyecto individual, los roles de Scrum se han repartido de la manera más lógica posible:

- **Product Owner:** Los tutores del TFG (José Enrique Sánchez López y Pablo Reina Jiménez), encargados de definir prioridades y validar que lo desarrollado tenía sentido.

- **Scrum Master y Equipo de Desarrollo:** Yo mismo, Kevin Amador Calzadilla, que gestionaba el backlog, ejecutaba los sprints y hacía las retrospectivas (básicamente, pararme a pensar qué había ido bien y qué podía mejorar).

3.2.2. Artefactos

- **Product Backlog:** Lista priorizada de funcionalidades y tareas técnicas, mantenida y actualizada de forma continua a lo largo del desarrollo.
- **Sprint Backlog:** Subconjunto de tareas seleccionadas para cada sprint de dos semanas.
- **Incremento:** Cada sprint concluye con una versión funcional del sistema que integra las nuevas funcionalidades desarrolladas.

3.2.3. Ceremonias

Las ceremonias se adaptaron al formato individual: al inicio de cada sprint me sentaba a planificar qué tareas abordaría en las dos semanas siguientes, y al finalizar revisaba los avances con los tutores y hacía una retrospectiva personal para identificar qué mejorar en el siguiente ciclo.

3.3. Herramientas de gestión

- **Control de versiones:** Git con repositorio alojado en GitHub.
- **Gestión de tareas:** Seguimiento manual del backlog y los sprints mediante documentación propia del alumno.
- **Entorno de desarrollo:** Visual Studio Code con extensiones para Python, LaTeX y Docker.
- **Contenedorización:** Docker y Docker Compose para la reproducibilidad del entorno.

Capítulo 4

Planificación

4.1. Definición de sprints

El trabajo se organizó en sprints de dos semanas, cada uno con un hito principal bien definido. A continuación se muestra cómo se distribuyeron:

Tabla 4.1: Planificación de sprints del proyecto

Sprint	Periodo	Hito principal
1	Semanas 1–2	Investigación del estado del arte y definición de requisitos
2	Semanas 3–4	Pipeline de ingesta: extracción de PDFs y fragmentación
3	Semanas 5–6	Implementación del RAG básico con Pinecone y Cohere
4	Semanas 7–8	Diseño e implementación del grafo LangGraph
5	Semanas 9–10	Desarrollo del backend FastAPI y modelo de datos
6	Semanas 11–12	Desarrollo del frontend Streamlit
7	Semanas 13–14	Migración a Groq y optimización de latencia
8	Semanas 15–16	Contenedorización con Docker Compose
9	Semanas 17–18	Evaluación y validación del sistema
10	Semanas 19–20	Redacción de la memoria y documentación final

4.2. Diagrama de Gantt

Para visualizar mejor la distribución temporal de las tareas y sus dependencias, se ha elaborado un diagrama de Gantt que refleja el cronograma seguido durante las 20 semanas de desarrollo.

Nota: Inserta aquí la imagen de tu diagrama de Gantt. Puedes usar el siguiente bloque como plantilla:

Figura 4.1: Diagrama de Gantt de la planificación del TFG

4.3. Estimación de esfuerzo

Se estima una dedicación media de 15–20 horas semanales durante 20 semanas, totalizando aproximadamente 300–400 horas de trabajo. La distribución aproximada del esfuerzo por área es:

- Investigación y estado del arte: 15 %
- Diseño e implementación del pipeline RAG: 25 %
- Desarrollo del agente (LangGraph): 20 %
- Backend y frontend: 15 %
- Evaluación y validación: 10 %
- Documentación y memoria: 15 %

4.4. Análisis de riesgos

A lo largo del ciclo de vida del proyecto, se han identificado diversos riesgos que podrían comprometer la planificación o la calidad del sistema. Para cada uno de ellos se ha evaluado su probabilidad de ocurrencia y su impacto en el proyecto, definiendo estrategias de mitigación proactivas.

Tabla 4.2: Matriz de análisis de riesgos del proyecto

Riesgo	Probabilidad	Impacto	Mitigación
Límite de tokens en APIs gratuitas (Groq, Llama-Parse)	Alta	Alto	Estrategia multi-modelo con modelos ligeros para tareas simples
Alucinaciones del LLM	Media	Alto	Temperatura baja (0.1) y RAG estricto con fuentes verificables
Mala fragmentación de tablas en PDFs	Alta	Medio	Migración a LlamaParse con prompts especializados
Latencia excesiva en respuestas	Media	Medio	Migración de endpoints de HuggingFace a Groq (LPUs)
Pérdida de contexto en conversaciones largas	Baja	Bajo	Implementación de ventana deslizante y resumen de memoria
Curva de aprendizaje de LangGraph	Media	Medio	Asignación de tiempo extra en los primeros sprints de diseño

4.5. Estimación de costes

Aunque este proyecto es puramente académico, resulta interesante hacer el ejercicio de estimar cuánto costaría si se tratase de un encargo profesional real desarrollado por un ingeniero de software junior. He dividido la estimación en tres partidas: personal, recursos materiales y costes indirectos.

4.5.1. Costes de personal

Tomando como referencia la estimación de 400 horas de dedicación calculada en la sección anterior, y asumiendo un coste por hora estándar para un perfil de desarrollador junior de 15 €/hora, el coste de personal asciende a:

- **Desarrollador Junior:** $400 \text{ horas} \times 15 \text{ €/hora} = 6.000 \text{ €}$

4.5.2. Costes de hardware y software

- **Hardware:** Se ha empleado un equipo informático portátil valorado en 1.200 €. Considerando un periodo de amortización de 4 años, y dado que el proyecto ha durado 5 meses, la cuota de amortización imputable al proyecto es de aproximadamente 125 €.

- **Software y APIs:** El desarrollo se ha apoyado mayoritariamente en herramientas de código abierto (Python, FastAPI, Streamlit, Docker). Los servicios de IA y bases de datos vectoriales (Groq, Cohere, Pinecone, LlamaParse) se han utilizado en su capa gratuita (*Free Tier*), por lo que el coste directo en este apartado es de 0 €. Sin embargo, en un entorno de producción, los costes de inferencia y almacenamiento escalarían según el uso.

4.5.3. Costes indirectos

Se contemplan gastos de suministros como electricidad y conexión a internet durante los 5 meses de desarrollo, que se estiman conservadoramente en 20 € mensuales imputables al proyecto.

- **Suministros:** 5 meses $\times$ 20 €/mes = 100 €

4.5.4. Coste total del proyecto

Sumando todas las partidas anteriores, obtenemos el coste total estimado del proyecto:

Tabla 4.3: Presupuesto total estimado

Concepto	Coste estimado (€)
Costes de Personal	6.000,00
Costes de Hardware (Amortización)	125,00
Costes de Software y APIs	0,00
Costes Indirectos	100,00
Base Imponible	6.225,00
IVA (21 %)	1.307,25
Total Presupuesto	7.532,25

Parte III

Análisis y Diseño

Capítulo 5

Análisis de requisitos

5.1. Requisitos funcionales

Tabla 5.1: Requisitos funcionales del sistema

ID	Descripción
RF-01	El sistema debe permitir el registro de usuarios mediante email y contraseña.
RF-02	El sistema debe permitir la autenticación de usuarios mediante JWT.
RF-03	El usuario debe poder crear, renombrar y eliminar conversaciones.
RF-04	El sistema debe procesar la consulta del usuario y generar una respuesta fundamentada en documentos oficiales.
RF-05	El sistema debe clasificar la intención del usuario y enrutar la consulta al nodo especializado correspondiente.
RF-06	El sistema debe rechazar amablemente consultas que no estén relacionadas con trámites de la US.
RF-07	El sistema debe solicitar aclaraciones al usuario cuando la consulta sea ambigua.
RF-08	El sistema debe mostrar las fuentes y referencias consultadas junto con cada respuesta.
RF-09	El sistema debe mantener el contexto de la conversación a lo largo de múltiples turnos.
RF-10	Los administradores deben poder ingestar nuevos documentos PDF al sistema desde la interfaz.
RF-11	El sistema debe generar automáticamente títulos descriptivos para las conversaciones.

Tabla 5.1: Requisitos funcionales del sistema (continuación)

ID	Descripción
RF-12	El sistema debe comprimir la memoria de conversaciones largas para mantener la coherencia.
RF-13	El sistema debe mostrar a los administradores las cuentas de usuario existentes en el programa.
RF-14	El sistema debe permitir a los administradores eliminar cuentas de usuario.
RF-15	El sistema debe permitir a los administradores convertir a otros usuarios en administradores.
RF-16	El sistema debe permitir a los administradores degradar a otros administradores en usuarios.

5.2. Requisitos no funcionales

Tabla 5.2: Requisitos no funcionales del sistema

ID	Descripción
RNF-01	El tiempo de respuesta del agente no debe superar los 20 segundos.
RNF-02	El sistema debe desplegarse mediante contenedores Docker para garantizar la portabilidad.
RNF-03	Las contraseñas deben almacenarse hasheadas con bcrypt.
RNF-04	La interfaz debe ser accesible desde cualquier navegador web moderno.
RNF-05	El sistema debe ser capaz de procesar documentos PDF de hasta 100 páginas.
RNF-06	Las respuestas del agente deben basarse exclusivamente en los documentos ingestados (sin alucinaciones).
RNF-07	La arquitectura debe permitir la adición de nuevos tipos de nodos sin modificar el núcleo del sistema.

5.3. Casos de uso

Los principales casos de uso identificados son:

- **CU-01: Registrarse.** Un usuario nuevo crea una cuenta proporcionando email y contraseña.
- **CU-02: Iniciar sesión.** Un usuario registrado se autentica para acceder al sistema.
- **CU-03: Realizar consulta burocrática.** El usuario formula una pregunta sobre normativa o trámites de la US y recibe una respuesta fundamentada.
- **CU-04: Gestionar conversaciones.** El usuario crea, renombra o elimina sus conversaciones.
- **CU-05: Ingestar documento (Admin).** Un administrador sube un nuevo PDF para ampliar la base de conocimiento del agente.
- **CU-06: Gestionar usuarios (Admin).** Un administrador puede gestionar usuarios ya sea añadiendo como nuevo administrador, eliminándolo, degradándolo a usuario normal.

Capítulo 6

Diseño de la arquitectura

6.1. Arquitectura general del sistema

El sistema sigue una arquitectura de microservicios contenedorizados orquestados mediante Docker Compose. Los tres servicios principales son:

- **Frontend (Streamlit):** Interfaz web que se comunica con el backend vía HTTP REST. Puerto 8501.
- **Backend (FastAPI):** API REST que gestiona la autenticación, las conversaciones y orquesta el agente LangGraph. Puerto 8000.
- **Base de datos (PostgreSQL 15):** Almacena usuarios, conversaciones y mensajes. Puerto 5432.

Adicionalmente, el sistema se conecta con servicios externos en la nube:

- **Pinecone:** Base de datos vectorial para almacenamiento y búsqueda de embeddings.
- **Groq:** API de inferencia para los modelos Llama 3.1 8B y Llama 3.3 70B.
- **Cohere:** API para generación de embeddings y reranking de documentos.
- **LlamaParse:** API para el procesamiento inteligente de PDFs.

6.2. Arquitectura del backend (FastAPI)

6.2.1. Modelo de datos

El modelo relacional del sistema se compone de tres entidades principales implementadas con SQLAlchemy ORM:

- **Usuario:** Almacena el email, contraseña hasheada, rol de administrador y fecha de creación. Relación uno a muchos con Conversación.
- **Conversación:** Contiene el título (auto-generado), un campo de resumen de memoria para conversaciones largas y la referencia al usuario propietario. Relación uno a muchos con Mensaje.
- **Mensaje:** Almacena el rol (user/assistant), el contenido textual, las referencias a documentos consultados (en formato JSON) y la marca temporal.

Las relaciones incluyen eliminación en cascada: al eliminar un usuario se eliminan sus conversaciones, y al eliminar una conversación se eliminan sus mensajes.

6.2.2. Endpoints de la API REST

Tabla 6.1: Endpoints principales de la API

Método	Ruta	Descripción
POST	/registro	Registro de nuevo usuario
POST	/login	Autenticación con email/contraseña
GET	/me	Datos del usuario autenticado
GET	/conversaciones	Lista de conversaciones del usuario
POST	/conversaciones	Crear nueva conversación
GET	/conversaciones/{id}/mensajes	Historial de mensajes
POST	/conversaciones/{id}/chat	Enviar mensaje y obtener respuesta del agente
PUT	/conversaciones/{id}	Renombrar conversación
DELETE	/conversaciones/{id}	Eliminar conversación
POST	/admin/ingestar	Subir PDF para ingesta (solo admin)

6.2.3. Autenticación y autorización (JWT)

El sistema implementa autenticación basada en tokens JWT (*JSON Web Tokens*) con el algoritmo HS256. Los tokens tienen una validez de 7 días. El flujo es: el usuario envía sus credenciales al endpoint `/login`, el servidor verifica la contraseña contra el hash bcrypt almacenado en la base de datos, y si es correcta, genera un token JWT que el cliente incluirá en la cabecera **Authorization** de las solicitudes posteriores.

Para las rutas de administración (como `/admin/ingestar`), se añade una capa adicional que verifica el campo `is_admin` del usuario autenticado.

6.3. Diseño del agente conversacional

6.3.1. Grafo de estados (LangGraph)

El agente se implementa como un `StateGraph` de `LangGraph` con ocho nodos y transiciones condicionales. El estado compartido entre nodos se define mediante un `TypedDict` (`StateSchema`) con los siguientes campos: `pregunta`, `pregunta_reformulada`, `historial`, `historial_formateado`, `contexto`, `stream` y `referencias`.

El flujo general del grafo es:

1. **START** → **Estado Inicial**: Formatea el historial de la conversación.
2. **Estado Inicial** → **Recuperador** — **Rechazo**: Decide si la consulta es relevante.
3. **Recuperador** → **Entrevistador** — **Clasificador**: Decide si hay suficiente información o se necesitan aclaraciones.
4. **Clasificador** → **Procedimental** — **Calendario** — **Normativo** — **Baremo**: Enruta al nodo especializado.
5. **Nodo final** → **END**: Genera la respuesta y finaliza.

6.3.2. Nodo Estado Inicial

Formatea el historial de mensajes de la conversación en un formato legible (“rol: contenido”) que será utilizado por los prompts de los nodos posteriores.

6.3.3. Nodo Router: clasificación de intenciones

Utiliza un LLM clasificador ultra-ligero (Llama 3.1 8B con `max_tokens=15`) para determinar si la consulta del usuario es relevante para el dominio del agente. Devuelve “recuperador” si la consulta está relacionada con trámites de la US, o “rechazo_amable” en caso contrario.

6.3.4. Nodo Recuperador: pipeline RAG

Este nodo ejecuta el pipeline completo de recuperación de información:

1. Reformula la pregunta del usuario integrando el contexto del historial.
2. Genera el embedding de la pregunta reformulada con Cohere.
3. Busca los 12 documentos más similares en Pinecone.
4. Aplica reranking con Cohere (`rerank-v4.0-pro`) seleccionando los 4 más relevantes.

5. Formatea los documentos con metadata (fuente y página).

6.3.5. Nodo Entrevistador: gestión de ambigüedades

Se activa cuando el sistema detecta que la consulta del usuario es ambigua o le faltan datos clave. El nodo genera una respuesta en dos partes: primero ofrece información general sobre el tema consultado, y al final formula una pregunta aclaratoria para obtener el dato que falta. Una regla estricta impide que el agente haga dos preguntas consecutivas al usuario.

6.3.6. Nodos Resolutores: respuestas especializadas

Se han diseñado cuatro nodos resolutores, cada uno con un prompt especializado:

- **Procedimental:** Genera respuestas paso a paso para trámites administrativos (matrícula, liquidación de viajes, etc.), usando listas numeradas y resaltando plataformas y documentos requeridos.
- **Calendario:** Especializado en fechas, plazos y periodos del calendario académico, con especial énfasis en la precisión de las fechas.
- **Normativo:** Explica normativas, requisitos legales y resoluciones rectorales, citando artículos específicos cuando están disponibles en el contexto.
- **Baremo:** Detalla criterios de evaluación, puntuaciones y méritos en procesos de selección de profesorado, utilizando tablas cuando es apropiado.

6.3.7. Nodo Rechazo Amable

Genera una respuesta educada y empática indicando al usuario que su consulta no puede ser atendida, sugiriendo alternativas cuando es posible.

6.4. Diseño del sistema Dual RAG

El sistema implementa dos niveles de RAG paralelos:

- **RAG de conocimiento normativo:** La base de datos vectorial en Pinecone contiene los fragmentos de los 15 documentos oficiales de la US (normativas de matrícula, baremos de contratación, calendarios académicos, guías de viajes, etc.).
- **RAG de buenas prácticas:** Los prompts de cada nodo resolutor incorporan directrices de estilo, formato y tono que guían la generación de respuestas cercanas al usuario sin sacrificar el rigor técnico.

De esta forma, el contenido factual proviene exclusivamente de los documentos oficiales, mientras que la forma y estructura de la respuesta se define mediante las instrucciones embebidas en los templates de cada nodo.

6.5. Diseño del frontend (Streamlit)

La interfaz web se organiza en tres áreas funcionales:

- **Pantalla de autenticación:** Tabs de inicio de sesión y registro, con el logo de la US.
- **Barra lateral:** Lista de conversaciones con opciones de crear, renombrar y eliminar. Para administradores, incluye un panel de gestión de conocimiento para ingestar nuevos PDFs.
- **Área de chat:** Burbujas de mensaje con animación de entrada (*fade-in*), streaming simulado de respuestas palabra por palabra, y expansores para mostrar las fuentes consultadas.

El diseño visual utiliza una paleta oscura con tipografía Inter, siguiendo una estética moderna tipo SaaS. Los botones cambian al color corporativo de la US (#9D1C34) al pasar el cursor.

Parte IV

Implementación

Capítulo 7

Infraestructura y despliegue

7.1. Contenedorización con Docker

7.1.1. Docker Compose: orquestación de servicios

El sistema se despliega mediante un archivo `docker-compose.yml` que define tres servicios:

- **db:** Contenedor PostgreSQL 15 con volumen persistente (`postgres_data`) para la durabilidad de los datos. Se expone en el puerto 5434 del host.
- **backend:** Contenedor con la aplicación FastAPI, que se construye desde el directorio `./backend`. Depende del servicio `db` y se expone en el puerto 8000.
- **frontend:** Contenedor con la aplicación Streamlit, construido desde la raíz del proyecto. Se conecta al backend mediante la URL interna `http://backend:8000` y se expone en el puerto 8501.

7.1.2. Configuración de red y variables de entorno

Docker Compose crea automáticamente una red interna que permite la comunicación entre servicios por nombre de host. Las credenciales de bases de datos, claves API (Groq, Pinecone, Cohere, LlamaParse) y parámetros de configuración se gestionan mediante un archivo `.env` compartido, evitando la exposición de secretos en el código fuente.

7.2. Base de datos PostgreSQL

Se utiliza PostgreSQL 15 como sistema de gestión de base de datos relacional. La interacción con la base de datos se realiza a través de SQLAlchemy ORM con la siguiente configuración:

- Motor de conexión con `client_encoding=utf8` para soporte de caracteres especiales en español.
- Sesiones gestionadas mediante un generador (`get_db`) que garantiza el cierre correcto de conexiones.
- Creación automática de tablas al iniciar la aplicación mediante `Base.metadata.create_all`.

Capítulo 8

Pipeline de ingesta de documentos

8.1. Procesamiento con LlamaParse

LlamaParse se configura con un `system_prompt` especializado en documentos burocráticos universitarios que instruye al modelo para:

1. Extraer todas las celdas de tablas sin omitir texto, incluyendo celdas combinadas.
2. Mantener notas al pie y aclaraciones inmediatamente después de la sección a la que hacen referencia.
3. Respetar la jerarquía de encabezados (H1, H2, H3) y listas enumeradas.
4. Extraer el texto íntegramente, sin resumir.

La extracción se realiza mediante el método `get_json_result`, que devuelve los resultados organizados por página, permitiendo asociar cada fragmento con su página de origen en los metadatos.

8.2. Fragmentación semántica (chunking)

8.2.1. Evolución de la estrategia de fragmentación

Si algo aprendí durante el desarrollo de este proyecto es que la forma de fragmentar los documentos importa muchísimo más de lo que parece a primera vista. De hecho, la estrategia de fragmentación evolucionó a lo largo de tres fases hasta dar con una solución satisfactoria:

1. **Fase inicial:** Fragmentación genérica con chunks de 1000 caracteres y solapamiento de 200. Los fragmentos carecían de contexto suficiente y las tablas quedaban rotas.

2. **Fase intermedia (Docling):** Se incorporó Docling para una extracción más inteligente. Mejoró la claridad del contexto pero seguía fallando con tablas y elementos complejos.
3. **Fase final (LlamaParse):** La migración a LlamaParse resolvió la preservación de tablas y jerarquías documentales, produciendo Markdown estructurado de alta calidad.

8.2.2. El problema del doble splitter

Este fue uno de esos errores sutiles que te vuelven loco. En un momento dado, las respuestas sobre baremos empezaron a degradarse sin razón aparente. Después de bastante tiempo depurando, descubrí que se estaban ejecutando dos procesos de fragmentación de forma redundante: LlamaParse ya producía fragmentos semánticos de calidad, pero añadí un `MarkdownTextSplitter` adicional que volvía a cortar esos fragmentos, rompiendo tablas y separando notas de su contexto.

La solución fue sencilla una vez identificado el problema: confiar en la segmentación de LlamaParse para la estructura semántica, y usar el `MarkdownTextSplitter` únicamente como red de seguridad para fragmentos que excedieran el tamaño máximo.

8.2.3. Configuración final: `MarkdownTextSplitter`

La configuración final del splitter utiliza:

- `chunk_size = 1500`: Tamaño suficiente para contener una sección completa con su contexto.
- `chunk_overlap = 250`: Solapamiento que garantiza continuidad entre fragmentos adyacentes.

Tras la fragmentación, se aplica `filter_complex_metadata` para limpiar metadatos incompatibles con Pinecone.

8.3. Generación de embeddings con Cohere

Se utiliza el modelo `embed-multilingual-v3.0` de Cohere para generar embeddings de 1024 dimensiones. Este modelo fue seleccionado por su rendimiento superior en textos en español frente a alternativas como los embeddings de HuggingFace, y por su capacidad de capturar matices semánticos en documentación normativa.

8.4. Almacenamiento en Pinecone

Los vectores se almacenan en un índice denominado `index-tfg` en Pinecone. La subida se realiza por lotes de 100 fragmentos para evitar timeouts y respetar los límites de la API. Cada vector almacena como metadata el nombre del documento fuente y el número de página, información que posteriormente se utiliza para generar las referencias en las respuestas.

Capítulo 9

Implementación del agente

9.1. Esquema de estado (StateSchema)

El estado compartido entre todos los nodos del grafo se define mediante un `TypedDict` de Python con los siguientes campos:

- `pregunta (str)`: La consulta original del usuario.
- `pregunta_reformulada (str)`: La consulta enriquecida con contexto del historial.
- `historial (list)`: Lista de mensajes previos en formato `{role, content}`.
- `historial_formateado (list)`: Historial en formato texto plano para inyección en prompts.
- `contexto (str)`: Fragmentos de documentos recuperados del RAG.
- `stream (str)`: Generador de la respuesta para streaming.
- `referencias (list)`: Lista de fuentes consultadas.

9.2. Sistema de prompts

Todos los prompts del sistema se centralizan en un módulo `templates.py`, lo que facilita su mantenimiento y evolución iterativa.

9.2.1. Prompt de detección de intención

Determina si la consulta del usuario está relacionada con trámites burocráticos de la US. Devuelve exclusivamente “recuperador” o “rechazo_amable”. Se ejecuta con el LLM clasificador (`max_tokens=15`) para minimizar latencia.

9.2.2. Prompt de reformulación

Reformula la última intervención del usuario incorporando toda la información aclarada en turnos anteriores, generando una consulta autónoma que pueda utilizarse para la búsqueda semántica sin pérdida de contexto.

9.2.3. Prompt de suficiencia de información

Analiza si el contexto recuperado contiene información suficiente para dar una respuesta directa o si es necesario preguntar al usuario. Implementa una “Regla de Oro” que prohíbe devolver “entrevistador” si la última intervención del asistente fue una pregunta, evitando bucles de clarificación.

9.2.4. Prompt clasificador de categorías

Clasifica la consulta en una de cuatro categorías: procedimental, calendario, normativo o baremo, basándose en el tipo de respuesta que necesita el usuario. Incluye ejemplos concretos para cada categoría (*few-shot prompting*).

9.2.5. Prompts de los nodos resolutores

Cada nodo resolutor cuenta con un prompt especializado que define el formato, tono y nivel de detalle esperado. Los prompts incluyen instrucciones específicas sobre el uso de listas, tablas, negritas y emojis según la categoría.

9.3. Pipeline de recuperación

9.3.1. Búsqueda semántica en Pinecone

El proceso de búsqueda se ejecuta en tres etapas con instrumentación de tiempos:

1. **Embedding de la consulta:** Se genera el vector de la pregunta reformulada usando Cohere.
2. **Búsqueda vectorial:** Se recuperan los 12 documentos más similares ($k=12$) mediante `similarity_search_by_vector` en Pinecone.
3. **Reranking:** Se aplica Cohere Rerank para reordenar y seleccionar los 4 documentos más relevantes.

9.3.2. Reranking con Cohere

El reranking con el modelo `rerank-v4.0-pro` de Cohere implementa un `ContextualCompressionRetriever` que actúa como una segunda capa de filtrado. Recibe los 12 documentos candidatos y los reordena según su relevancia semántica respecto a la consulta, seleccionando únicamente los 4 mejores (`top_n=4`). Esto mejora significativamente la precisión del contexto inyectado al LLM.

9.4. Gestión de memoria conversacional

9.4.1. Ventana deslizante de mensajes

Para evitar exceder la ventana de contexto de los modelos de Groq, se implementa una ventana deslizante que envía al agente únicamente los últimos 5 mensajes de la conversación. El historial completo se almacena en la base de datos, pero solo el subconjunto reciente se utiliza para la inferencia.

9.4.2. Compresión de memoria con resumen

Cuando una conversación supera los 5 mensajes, se ejecuta una tarea en segundo plano (`BackgroundTasks` de FastAPI) que utiliza el LLM ligero para generar un resumen comprimido del historial antiguo. Este resumen se almacena en el campo `resumen_memoria` de la conversación y se inyecta como mensaje de sistema al inicio del historial, preservando información clave (tipo de estudios, problema principal, fechas mencionadas) sin consumir tokens excesivos.

9.5. Generación automática de títulos

Al enviar el primer mensaje de una conversación con título “Nueva conversación”, se dispara una tarea asíncrona que utiliza el LLM ligero para generar un título descriptivo de 4-5 palabras basado en el contenido del mensaje.

Capítulo 10

Implementación del frontend

10.1. Interfaz de autenticación

La pantalla de autenticación presenta dos tabs (Iniciar Sesión y Registrarse) con el logo de la Universidad de Sevilla. Se utiliza la función `api_request` como capa de abstracción para todas las llamadas HTTP, que incluye automáticamente el token JWT en las cabeceras cuando el usuario está autenticado.

10.2. Panel de conversaciones

La barra lateral muestra la lista de conversaciones del usuario, con la conversación activa marcada con un indicador visual. Incluye opciones para crear nuevas conversaciones, renombrarlas o eliminarlas, así como un botón de cierre de sesión.

10.3. Interfaz de chat

La interfaz de chat implementa las siguientes funcionalidades:

- **Burbujas de mensaje** con animación CSS `fadeInUp` para una entrada suave.
- **Streaming simulado:** Las respuestas se muestran palabra por palabra con un delay de 15ms mediante `st.write_stream`, creando una experiencia de escritura en tiempo real.
- **Referencias expandibles:** Cada respuesta del asistente incluye un expansor con las fuentes y páginas consultadas.
- **CSS personalizado:** Paleta oscura (`#000000` / `#171A21`), tipografía Inter, y botones con hover en color corporativo (`#9D1C34`).

10.4. Panel de administración

Los usuarios con rol de administrador (`is_admin=True`) acceden a un panel adicional en la barra lateral que permite subir documentos PDF. La ingesta se ejecuta en segundo plano mediante `BackgroundTasks`, y el usuario recibe confirmación inmediata del encolamiento.

Capítulo 11

Selección y configuración de modelos

11.1. Evolución de la infraestructura de inferencia

11.1.1. Fase 1: Modelos locales (Llama 3.1 8B)

Al principio del proyecto, la idea era ejecutar todo en local usando Llama 3.1 8B a través de Ollama. El modelo cumplía bastante bien con las tareas de clasificación, pero los tiempos de inferencia eran completamente inaceptables: entre 30 y 60 segundos por cada llamada al modelo. Para un asistente que se supone que debe responder en tiempo real, esto era inviable. Además, la ejecución local hacía imposible cualquier tipo de despliegue más allá de mi propio ordenador.

11.1.2. Fase 2: API de HuggingFace

El siguiente paso fue migrar a los endpoints de inferencia de HuggingFace, lo que mejoró bastante los tiempos. Pero apareció otro problema que no había previsto: el *cold start*. La primera solicitud tras un rato de inactividad podía tardar hasta 30 segundos mientras la instancia se aprovisionaba. Era frustrante porque funcionaba bien en las pruebas seguidas, pero en cuanto dejabas de usar el sistema un rato, la primera respuesta tardaba una eternidad.

11.1.3. Fase 3: API de Groq

La solución definitiva llegó con la API de Groq, que utiliza un hardware especializado llamado LPU (*Language Processing Units*) diseñado específicamente para ejecutar modelos de lenguaje. El cambio fue drástico: los tiempos de respuesta bajaron a fracciones de segundo por invocación, sin ningún problema de inicio en frío. La única pega es que la capa gratuita tiene límites de tokens por minuto y por día, pero para el ámbito de este proyecto fue más que suficiente.

11.2. Estrategia multi-modelo

Para optimizar el consumo de tokens y la latencia, se implementó una estrategia de tres niveles de modelo:

11.2.1. LLM clasificador (Llama 3.1 8B Instant)

Configurado con `temperature=0.0` y `max_tokens=15`. Se utiliza exclusivamente para tareas de clasificación binaria (detección de intención, suficiencia de información, categorización). Su respuesta es una única palabra clave, lo que minimiza el consumo de tokens.

11.2.2. LLM ligero (Llama 3.1 8B Instant)

Configurado con `temperature=0.1` y `max_tokens=300`. Se emplea para reformulación de consultas, generación de títulos, compresión de memoria y respuestas del nodo entrevistador y rechazo amable. Al tratarse de un modelo de 8B parámetros, ofrece baja latencia con calidad suficiente para estas tareas.

11.2.3. LLM principal (Llama 3.3 70B Versatile)

Configurado con `temperature=0.1` y `max_tokens=1500`. Se reserva para los nodos resolutores, donde la calidad de la respuesta es crítica. El modelo de 70B parámetros demuestra una capacidad cognitiva superior para interpretar normativa compleja y sintetizar información de múltiples fragmentos.

11.3. Ajuste de hiperparámetros

De todos los parámetros con los que experimenté, la `temperature` fue sin duda el más determinante. Con valores altos (0.7 o más), el modelo alucinaba con frecuencia preocupante; con 0.0, las respuestas eran correctas pero sonaban demasiado mecánicas. Al final me quedé con 0.1, que era el punto justo en el que el agente priorizaba la precisión factual pero seguía siendo capaz de parafrasear y adaptar su tono según el contexto. También configuré `max_retries` a 10 para el modelo principal, como colchón ante los posibles errores de *rate-limiting* de la API de Groq.

Parte V

Evaluación y Validación

Capítulo 12

Estrategia de evaluación

12.1. Dataset de evaluación

Para poder evaluar el sistema de forma rigurosa, diseñé un dataset con 34 preguntas que cubren todos los nodos del agente. La idea era simular consultas reales que podría hacer un estudiante, un profesor o un candidato a una plaza. Las preguntas se distribuyeron de la siguiente manera:

Tabla 12.1: Distribución del dataset de evaluación por categoría

Categoría	N. ^o de preguntas
Calendario	8
Procedimental	7
Normativo	7
Baremo	5
Consulta (ambigua)	2
Rechazo	5
Total	34

Cada caso de prueba incluye: la pregunta del usuario, la ruta esperada del router y un *ground truth* que describe la respuesta correcta esperada.

12.2. Métricas empleadas

12.2.1. Routing Accuracy

Mide el porcentaje de consultas en las que el agente clasifica correctamente la intención del usuario y la enruta al nodo especializado adecuado. Es una métrica binaria: acierto (1) o fallo (0) por cada caso.

12.2.2. Faithfulness

Evalúa en una escala de 1 a 5 si la respuesta generada se basa exclusivamente en la información presente en el contexto recuperado, sin inventar datos. Una puntuación de 5 indica que toda la respuesta está fielmente soportada por los documentos.

12.2.3. Answer Relevancy

Evalúa en una escala de 1 a 5 si la respuesta del agente aborda adecuadamente la pregunta del usuario. Una puntuación de 5 indica que la respuesta responde perfectamente y de forma completa a lo preguntado.

12.2.4. Correctness

Compara la respuesta generada contra el *ground truth* definido, evaluando en una escala de 1 a 5 la coincidencia factual entre ambas. Una puntuación de 5 indica concordancia plena.

12.3. Evaluación con LLM como juez

Se adoptó el paradigma *LLM-as-Judge*, utilizando Llama 3.1 ejecutado localmente mediante Ollama como evaluador automático. Este enfoque evita el consumo de tokens de APIs externas durante la fase de evaluación.

El proceso de evaluación para cada caso de prueba sigue este pipeline:

1. El agente procesa la pregunta recorriendo el grafo completo (detección → recuperación → clasificación → generación).
2. Se registra la ruta tomada y se compara con la esperada (Routing Accuracy).
3. El LLM juez evalúa la respuesta generada contra el contexto (Faithfulness), contra la pregunta (Relevancy) y contra el ground truth (Correctness).
4. Los resultados se exportan a un archivo CSV para análisis posterior.

Capítulo 13

Resultados

13.1. Precisión del enrutamiento

Uno de los resultados que más me sorprendió fue la fiabilidad del sistema a la hora de clasificar las intenciones del usuario. Al asignar el rol de enrutador a Llama 3.1 8B con una temperatura de 0.0, conseguimos decisiones prácticamente deterministas en milisegundos. El nodo de rechazo, en particular, funcionó de forma especialmente consistente, interceptando sin problemas las preguntas que se salían de la temática universitaria.

Nota: Completa la siguiente tabla con los resultados numéricos de tus evaluaciones una vez hayas ejecutado los tests finales. Es muy recomendable incluir gráficos de barras o pastel visualizando estos resultados.

Tabla 13.1: Resultados de la precisión de enrutamiento

Categoría	Casos de Prueba	Aciertos (%)
Procedimental	7	X %
Calendario	8	X %
Normativo	7	X %
Baremo	5	X %
Rechazo	5	X %
Ambigüedad	2	X %
Total	34	XX.X %

13.2. Análisis de fidelidad (Faithfulness)

En general, el agente se ciñó bastante bien al contexto recuperado, lo que resulta lógico si tenemos en cuenta que la temperatura estaba configurada a 0.1 y que los prompts incluían instrucciones explícitas para no inventar nada. Los pocos casos con menor pun-

tuación coincidieron con consultas donde el contexto recuperado era solo parcialmente relevante, y el modelo —quizá por inercia— rellenaba los huecos con información general que sonaba plausible pero que no estaba en los documentos.

13.3. Análisis de relevancia

La relevancia de las respuestas fue consistentemente alta, especialmente en los nodos procedimental y calendario donde las instrucciones de formato (pasos numerados, fechas en negrita) facilitan respuestas directas y accionables.

13.4. Análisis de corrección

La corrección frente al *ground truth* fue la métrica más variable, y tiene su explicación: algunos *ground truths* los redacté de forma genérica (por ejemplo, “el plazo de matrícula está en septiembre”), mientras que el agente, al tener acceso directo a los documentos, proporcionaba datos mucho más específicos (fechas exactas, artículos concretos). Paradójicamente, en algunos casos el agente era más preciso que mi propio *ground truth*.

13.5. Comparativa: entorno local vs Groq

Tabla 13.2: Comparativa entre entorno local y Groq

Aspecto	Local (Llama 3.1 8B)	Groq (Llama 3.3 70B)
Latencia por nodo	>10 segundos	<1 segundo
Precisión en clasificación	Alta	Alta
Razonamiento complejo	Limitado	Superior
Flujos multi-turno	Finalizan abruptamente	Fluidos y naturales
Coste	Gratuito (hardware propio)	Gratuito (cuotas limitadas)
Escalabilidad	No viable	Viable

La migración a Groq fue, sin duda, el punto de inflexión del proyecto. Lo que antes era un prototipo con el que tenías que armarte de paciencia para cada consulta, se transformó en un sistema que respondía en menos de 10 segundos de extremo a extremo.

13.6. Evaluación del retriever (Hit Rate)

Más allá de la generación de texto, es vital evaluar el motor de búsqueda. Para ello, medimos la métrica *Hit Rate*, que indica la capacidad de la base de datos vectorial (Pi-

necone) para incluir el documento exacto que responde a la consulta dentro del top-4 de resultados devueltos al modelo.

Nota: Añade aquí los porcentajes de Hit Rate obtenidos en tus pruebas de recuperación. Si tienes una gráfica de "Hit Rate vs. Top-K", este es el lugar ideal para insertarla.

Parte VI

Conclusiones y Trabajo Futuro

Capítulo 14

Conclusiones

14.1. Cumplimiento de objetivos

Mirando hacia atrás, creo que el proyecto ha cumplido de forma satisfactoria los objetivos que se plantearon al inicio:

- **Ingesta semántica:** Se ha desarrollado un pipeline capaz de procesar 15 documentos oficiales de la US, preservando tablas, jerarquías y notas al pie mediante LlamaParse.
- **Enrutamiento inteligente:** El agente LangGraph clasifica correctamente las consultas y las dirige al nodo especializado correspondiente, con una alta precisión de enrutamiento.
- **Sistema Dual RAG:** Se ha implementado la separación entre conocimiento normativo (Pinecone) y directrices de estilo (prompts), logrando respuestas técnicamente correctas con un tono cercano al usuario.
- **Latencia optimizada:** La migración a Groq y la estrategia multi-modelo han reducido los tiempos de respuesta por debajo del umbral de 20 segundos establecido.
- **Validación:** Se ha evaluado el sistema con 34 casos de prueba utilizando el paradigma LLM-as-Judge.

14.2. Lecciones aprendidas

Más allá del resultado técnico, hay varias lecciones que me llevo de este trabajo y que creo que merece la pena compartir:

- **La ingesta importa más que el modelo:** Perdí bastante tiempo al principio intentando mejorar las respuestas cambiando de modelo, cuando el verdadero pro-

blema estaba en cómo fragmentaba los documentos. Una mala segmentación invalida cualquier mejora que hagas aguas abajo.

- **La temperatura es el hiperparámetro más crítico:** Parece un detalle menor, pero pasar de 0.7 a 0.1 fue la diferencia entre un agente que alucinaba con frecuencia y uno que se ceñía a los documentos. El valor de 0.1 resultó ser el punto justo entre precisión factual y cierta naturalidad en la redacción.
- **Los prompts son código:** La ingeniería de prompts requiere el mismo rigor iterativo que el desarrollo de software. Pequeños cambios en una palabra pueden alterar por completo el comportamiento del agente.
- **No todo necesita el modelo grande:** Usar Llama 3.3 70B para tareas de clasificación binaria era como matar moscas a cañonazos. La estrategia multi-modelo no solo reduce costes, sino que también baja la latencia de forma significativa.

14.3. Dificultades encontradas

No todo fue un camino de rosas, ni mucho menos. Estas fueron las dificultades más relevantes:

- **Procesamiento de tablas en PDFs:** Sin duda, el mayor dolor de cabeza técnico. Hicieron falta tres intentos distintos (fragmentación genérica → Docling → Llama-Parse) hasta conseguir que las tablas se preservaran correctamente.
- **Límites de APIs gratuitas:** Trabajar con las capas gratuitas de Groq, Cohere y LlamaParse fue un ejercicio constante de optimización. Cada token contaba, y hubo días en los que agoté la cuota de la API a mitad de una sesión de pruebas.
- **Bucles de clarificación:** Al principio, el nodo entrevistador tendía a hacer preguntas una y otra vez sin avanzar. Se resolvió implementando lo que llamé la “Regla de Oro” en el prompt de suficiencia: si el asistente acaba de preguntar, no puede volver a preguntar en el siguiente turno.
- **Redundancia del doble splitter:** Un error sutil que estuvo degradando las respuestas sobre baremos durante semanas hasta que lo localicé analizando el pipeline completo paso a paso.

Capítulo 15

Trabajo futuro

15.1. Mejoras en el pipeline RAG

Una de las líneas que más me gustaría explorar es la implementación de *Hypothetical Document Embeddings* (HyDE). La idea consiste en generar un documento hipotético a partir de la pregunta del usuario antes de lanzar la búsqueda en la base vectorial, lo que en teoría mejoraría bastante la precisión semántica de la recuperación.

15.2. Ampliación de la base de conocimiento

El sistema está pensado para escalar sin demasiado esfuerzo. La funcionalidad de ingesta desde el panel de administración ya permite añadir nuevos documentos sin tocar una línea de código. Una extensión natural sería incorporar documentos adicionales como planes de estudio, guías docentes o la normativa específica de TFG.

15.3. Integración con canales de mensajería

Otra vía que creo que tiene mucho potencial es conectar el agente con plataformas de mensajería como Telegram o WhatsApp. Al fin y al cabo, si el objetivo es que el usuario pueda resolver sus dudas de la forma más cómoda posible, tiene todo el sentido que pueda hacerlo directamente desde la aplicación que ya usa a diario.

15.4. Despliegue en producción

Para llevar este sistema a un entorno real, la idea sería migrarlo a un servicio cloud (AWS, Google Cloud o Azure) con balanceo de carga y escalado automático. También sería necesario añadir monitorización de métricas en tiempo real y algún mecanismo para

que los propios usuarios puedan reportar respuestas incorrectas, cerrando así el ciclo de mejora continua.

Bibliografía

- [1] Amazon Web Services. ¿Qué es un modelo de lenguaje grande (LLM)? Disponible en: <https://aws.amazon.com/es/what-is/large-language-model/>
- [2] LangChain AI. LangChain Documentation. Disponible en: <https://python.langchain.com/>
- [3] LangGraph Documentation. Disponible en: <https://langchain-ai.github.io/langgraph/>
- [4] RAGAS Framework. Automated Evaluation of RAG. Disponible en: <https://docs.ragas.io/>
- [5] Pinecone. Vector Database Documentation. Disponible en: <https://docs.pinecone.io/>
- [6] Cohere. Embeddings and Reranking API. Disponible en: <https://docs.cohere.com/>
- [7] Groq. LPU Inference Engine. Disponible en: <https://groq.com/>
- [8] LlamaIndex. LlamaParse Documentation. Disponible en: <https://docs.llamaindex.ai/>
- [9] FastAPI. Modern web framework for building APIs. Disponible en: <https://fastapi.tiangolo.com/>
- [10] Streamlit. Framework for data apps. Disponible en: <https://docs.streamlit.io/>
- [11] Docker. Container platform documentation. Disponible en: <https://docs.docker.com/>
- [12] Vaswani, A. et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*.
- [13] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

- [14] Meta AI. Llama 3: Open Foundation Language Models. Disponible en: <https://ai.meta.com/llama/>

Apéndice A

Batería de validación completa

A continuación, se detalla la lista exhaustiva de los 34 casos de prueba utilizados para someter al agente a evaluación, incluyendo la consulta simulada, la categoría esperada y la salida obtenida.

*Nota: Aquí deberás pegar o importar el archivo CSV/JSON con tus 34 preguntas. Puedes formatearlo como una tabla **longtable** o una lista numerada.*

Apéndice B

Prompts del sistema

Para asegurar la reproducibilidad de los resultados, se adjuntan en este anexo los prompts exactos (plantillas de sistema) utilizados en los distintos nodos de LangGraph.

Nota: Pega aquí el contenido de tu archivo `templates.py` utilizando el entorno `lstlisting` para formatearlo como código.

Apéndice C

Uso de la Inteligencia Artificial

En cumplimiento con las directrices académicas, se declara el uso de modelos de lenguaje para la asistencia en la generación de código LaTeX y el refactor de scripts de evaluación.